



M O N G O D B C O U R S E

MongoDB

Beginner to Interview Ready

A complete beginner-friendly journey through MongoDB —
from the document model to cracking real interview questions.

Taught with clarity. Built for jobs.

Your Learning Roadmap

Eight focused modules — each building on the previous

01

Meet MongoDB

What it is, when to use it

02

Terminology & Types

The vocabulary of documents

03

Tools of the Trade

Atlas, Compass, VS Code, Server

04

CRUD & Queries

Insert, find, update, delete

05

Schema Design

Embed vs reference, patterns

06

Indexing

Speed, ESR, explain()

07

Performance & Transactions

Concerns, profiler, ACID

08

Security & MERN

Encryption, driver usage

MODULE 01

Meet MongoDB

What it is • Why it exists • When to use it

What is MongoDB?

A document database built for developer productivity

MongoDB is a **NoSQL document database** that stores data as flexible, JSON-like documents instead of rigid rows and columns.



Document-based



Highly scalable



Flexible schema

A document looks like this:

```
{
  "_id": ObjectId(...),
  "name": "Aisha Khan",
  "age": 27,
  "skills": ["js", "react"],
  "address": {
    "city": "Dhaka"
  }
}
```

INTERVIEW TIP Q: "Is MongoDB a NoSQL database?"

Yes — it's a document-oriented NoSQL database. Data lives in flexible BSON documents grouped into collections, not rows and tables.

MongoDB vs SQL Databases

Two philosophies, two strengths

SQL (Relational)

MongoDB (Document)

Data stored in	Tables → Rows → Columns	Databases → Collections → Documents
Schema	Rigid, enforced upfront	Flexible, evolves with your app
Relationships	JOINS across tables	Embed or reference (mostly embed)
Scaling	Vertical (bigger servers)	Horizontal (sharding built-in)
Best for	Tx-heavy: banking, ERP, inventory	Content, catalogs, real-time apps

INTERVIEW TIP Q: "When would you choose MongoDB over SQL?"

When your data is document-shaped, schema may evolve, reads need nested data in one go, or you need to scale horizontally. Choose SQL for strict relational integrity and multi-table transactions.

The Document Model Philosophy

Data that travels together, lives together

The Core Idea

Most applications work with **objects**. MongoDB stores data the same way your code thinks about it — as nested, self-contained objects.

"Data that is accessed together should be stored together."

SQL: data spread across tables

users	
id	name

orders	
id	user_id

items	
id	order_id

→ Needs JOINS across 3 tables

MongoDB: everything in one document

```
{ "user": "Aisha",  
  "orders": [  
    { "items": ["book", "pen"] }  
  ]  
} // one read, no joins
```

INTERVIEW TIP Q: "Why does MongoDB not need joins as often?"

Because related data that's read together is embedded inside the same document. One query returns everything — faster reads, simpler code.

MODULE 02

Terminology & Data Types

Learn the vocabulary you'll use every day

MongoDB Terminology

Think of it as folders inside folders



Database

A container that holds many collections. One app usually = one database.

```
shopDB
```



Collection

A group of related documents. Like a table in SQL, but without a fixed schema.

```
products
```



Document

One record, stored as BSON (binary JSON). Like a row in SQL.

```
{ name: 'Pen', price: 25 }
```



Field

A key-value pair inside a document. Like a column in SQL.

```
price: 25
```


BSON vs JSON

Same idea, different packaging

JSON

JavaScript Object Notation — the text format you already know

- ✓ Human-readable text
- ✓ Universal — every language reads it
- ✗ Only basic types (string, number, bool, null, array, object)
- ✗ No native Date, no big integers
- ✗ Larger size, slower to parse at scale

BSON

Binary JSON — what MongoDB actually stores on disk

- ✓ Binary encoding — fast to parse & traverse
- ✓ Extra types: Date, ObjectId, Decimal128, Binary
- ✓ Stores field length → fast skipping
- ✓ Supports 64-bit integers and true decimals
- **You write JSON, MongoDB stores BSON**

INTERVIEW TIP Q: "Does MongoDB store data as JSON?"

No — it stores data as BSON, a binary form of JSON. BSON adds extra types (Date, ObjectId, Decimal128) and is faster to parse and traverse. You interact with it as JSON-like documents.

Common BSON Data Types

The building blocks of every document

String

UTF-8 text

"Dhaka"

Decimal128

Exact decimals (money!)

NumberDecimal("19.99")

ObjectId

12-byte unique id, auto-generated

ObjectId("6523...")

Null

Explicit absence of value

null

Int32 / Int64

Whole numbers

42

Boolean

true or false

true

Array

Ordered list

["js", "python"]

Double

Floating point

3.14

Date

UTC datetime (not string)

ISODate("2026-04-16")

Object

Embedded document (nested)

{ city: "Dhaka" }

INTERVIEW TIP Q: "Why use Decimal128 instead of Double for prices?"

Double uses binary floating point → tiny rounding errors. Decimal128 stores exact base-10 decimals, critical for money, billing, and anything financial.

ObjectId — MongoDB's Default ID

12 bytes of automatic uniqueness

Every document gets an `_id` field automatically. By default, MongoDB sets it to an ObjectId — a 12-byte value guaranteed to be unique across machines and time.



Example:

```
ObjectId("652abc123f9d4e2a1bc7")
```

Green = timestamp • Light green = random • Amber = counter

INTERVIEW TIP Q: "Can you get the creation time from an `_id`?"

Yes — the first 4 bytes of an ObjectId store the timestamp. Most drivers expose `.getTimestamp()` on the ObjectId. It's a handy "free" `created_at` for many apps.

MODULE 03

Tools of the Trade

Where you actually work with MongoDB every day

The MongoDB Toolkit

Four tools. Pick the right one for the job.



Atlas

CLOUD

Fully-managed MongoDB in the cloud. Free tier available. What most companies actually use in production.



Community Server

LOCAL

Install MongoDB on your own machine. Best for offline learning and complete control.



Compass

GUI

The official desktop GUI. Point-and-click browsing, query building, and index management.



VS Code Extension

EDITOR

MongoDB for VS Code. Connect, browse, run queries, and build aggregations inside your editor.

When to Use Which Tool?

A quick decision guide

I'm starting out and want zero setup	→	Atlas free tier
I want to work offline on my laptop	→	Community Server
I want to visually browse my data	→	Compass
I'm writing code and want to run queries fast	→	VS Code Extension
I'm deploying a production app	→	Atlas (managed cloud)
I want to learn indexing & explain plans	→	Compass + Atlas

MODULE 04

CRUD & Queries

The four operations you'll do 1000 times a day

The Four CRUD Operations

Create. Read. Update. Delete.

C

CREATE

Add new documents

```
insertOne()  
insertMany()
```

R

READ

Query documents

```
find()  
findOne()
```

U

UPDATE

Modify documents

```
updateOne()  
updateMany()  
replaceOne()
```

D

DELETE

Remove documents

```
deleteOne()  
deleteMany()
```


CREATE — insertOne & insertMany

How to add data to a collection

insertOne() — add a single document

```
db.users.insertOne({
  name: "Rahim",
  age: 28,
  city: "Dhaka"
})
```

→ If `_id` is not provided, MongoDB generates an `ObjectId` automatically.

insertMany() — add multiple at once

```
db.users.insertMany([
  { name: "Aisha", age: 27 },
  { name: "Karim", age: 32 },
  { name: "Fatima", age: 24 }
])
```

→ Faster than many `insertOne` calls. Use whenever you have an array.

INTERVIEW TIP Q: "What happens if insertMany fails halfway?"

By default, MongoDB stops at the first error (`ordered: true`). Already-inserted docs stay. Pass `{ ordered: false }` to continue inserting the rest even if some fail.

READ — find() & Projection

Ask for exactly what you need, nothing more

Anatomy of a find() call:

```
db.users.find( { age: { $gt: 25 } }, { name: 1, age: 1, _id: 0 } )
```

FILTER

which documents?

Common filter shortcuts

`find({})` → all documents
`find({ city: "Dhaka" })` → exact match
`findOne({ _id: ... })` → single doc
`find({ age: { $gt: 25 } })` → with operators

PROJECTION

which fields to return?

Projection rules

- 1 → include this field
- 0 → exclude this field
- Can't mix 1s and 0s (except `_id`)
- `_id` is included by default — set to 0 to hide

INTERVIEW TIP Q: "Why is projection important?"

It reduces network traffic and memory usage. A query that returns 10,000 docs with 50 fields each is wasteful if you only need 2 fields. Projection also enables "covered queries" (answered entirely from an index).

Query Operators

Your vocabulary for expressing conditions

Comparison

\$eq	equals
\$ne	not equals
\$gt	greater than
\$gte	greater or equal
\$lt	less than
\$lte	less or equal
\$in	in an array
\$nin	not in array

Logical

\$and	all must match
\$or	any must match
\$not	invert a condition
\$nor	none must match

Element & Evaluation

\$exists	field is present
\$type	field has this type
\$regex	pattern match
\$expr	use expressions

```
db.users.find({ $and: [ { age: { $gte: 18 } }, { city: { $in: ["Dhaka", "Sylhet"] } } ] })
```

Adults in Dhaka or Sylhet

Cursor Methods — Shape Your Results

Chain these onto find() to refine the output

.sort()

`.sort({ age: -1 })`

Order by field(s). 1 = asc, -1 = desc.

.limit()

`.limit(10)`

Cap how many documents come back.

.skip()

`.skip(20)`

Skip N docs — used for pagination.

**.count() /
.countDocuments()**

`.countDocuments()`

How many match? countDocuments is the modern choice.

.distinct()

`.distinct('city')`

Unique values of a field across all matches.

INTERVIEW TIP Q: "How do you paginate in MongoDB?"

Naively: `.skip(page*size).limit(size)` — simple but slow for deep pages. Better: use range queries on an indexed sort key (e.g., `{ _id: { $gt: lastId } }`) — this scales to millions of rows.

UPDATE — Modifying Documents

\$set, \$unset, \$inc, \$push, and friends

Three methods

`updateOne(filter, update)`

Updates the first match. Most common.

`updateMany(filter, update)`

Updates every match. Be careful!

`replaceOne(filter, newDoc)`

Replaces the WHOLE document (except `_id`).

Key update operators

<code>\$set</code>	set a field (create if missing)
<code>\$unset</code>	remove a field
<code>\$inc</code>	increment a number
<code>\$push</code>	append to an array
<code>\$addToSet</code>	push only if not already there
<code>\$pull</code>	remove matching items from array
<code>\$pop</code>	remove first (-1) or last (1) item

```
db.users.updateOne({ _id: 1 }, { $set: { city: "Sylhet" }, $inc: { loginCount: 1 } })
```

DELETE — Removing Documents

Simple, but dangerous. Double-check your filter.

deleteOne() — remove one document

```
db.users.deleteOne({
  _id: ObjectId("6523...")
})
```

Deletes the first match only.

deleteMany() — remove many

```
db.users.deleteMany({
  active: false
})
```

Deletes every match. No undo!

⚠ DELETE RULES

- Empty filter {} with deleteMany will wipe the entire collection — always test your filter with find() first.
- In production, prefer 'soft deletes' (setting isDeleted: true) over real deletions for important records.
- deleteOne and deleteMany return a count — always check it.

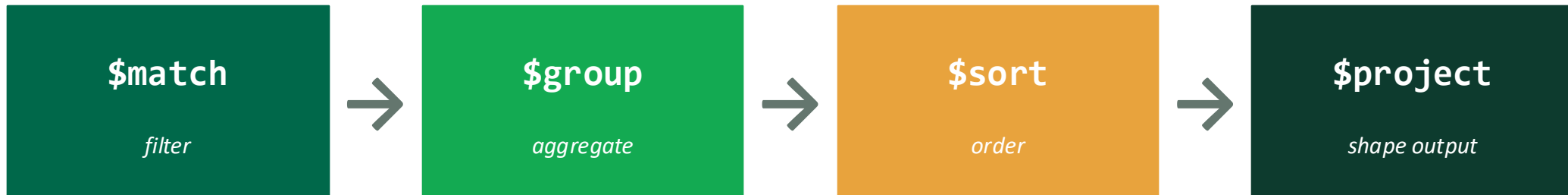
INTERVIEW TIP Q: "What's the difference between deleteOne and deleteMany?"

deleteOne removes exactly the first matching document and stops. deleteMany removes every match. Choose deleteOne whenever you logically expect a single target — it's safer if your filter has a bug.

Aggregation Pipeline — A Glimpse

A data assembly line

When you need to group, transform, or calculate — `find()` isn't enough. You use a pipeline of stages, where each stage transforms the documents and passes them to the next.



```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: { _id: "$city", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
])
```

INTERVIEW TIP Q: "When would you use aggregation instead of `find()`?"

When you need to group, compute sums/averages, join collections (`$lookup`), or reshape documents. `find()` reads; aggregation computes.

MODULE 05 ★ MOST IMPORTANT

Schema Design & Modeling

The one skill that separates juniors from seniors

Embed vs Reference

The #1 decision in every MongoDB design

EMBED (nest inside parent document)

```
{ "_id": 1, "name": "Aisha",  
  "address": { "city": "Dhaka" }  
}
```

- ✓ One query, one read
- ✓ Data read together lives together
- ✗ Document grows large if child is unbounded

REFERENCE (store an id, look up later)

```
// users  
{ _id: 1, name: "Aisha" }  
// orders  
{ _id: 99, userId: 1, amount: 450 }
```

- ✓ Handles unbounded / huge related data
- ✓ Avoid duplication when child is shared
- ✗ Needs a second query or \$lookup (slower)

INTERVIEW TIP Q: "When do you embed vs reference?"

Embed when child data is read with the parent, not shared, and bounded in size. Reference when child is huge, grows forever, or is shared by many parents (e.g., users ↔ orders).

Relationship Patterns

One-to-one, one-to-many, many-to-many in documents

One-to-One

e.g., User + Profile

Usually EMBED

```
{ _id: 1, name: "Aisha",  
  profile: { bio: "dev",  
            avatar: "..."}  
}
```

One-to-Few

e.g., User + Addresses (< ~100)

EMBED as array

```
{ _id: 1, name: "Karim",  
  addresses: [  
    { city: "Dhaka" },  
    { city: "Sylhet" }  
  ]  
}
```

One-to-Many

e.g., User + Orders (unbounded)

REFERENCE

```
// orders collection  
{ _id: 99,  
  userId: 1,  
  amount: 450 }
```

Many-to-Many

e.g., Students ↔ Courses

REFERENCE (array of ids)

```
// students  
{ _id: 1,  
  courseIds: [7, 12, 19]  
}
```

6 Rules of Thumb for Schema Design

MongoDB's official guidance, distilled

1

Favor embedding

Unless there is a compelling reason not to — it's faster and simpler.

2

Avoid needing to access an embedded object alone

If you query the child by itself often, make it its own collection.

3

Arrays should not grow unbounded

Hundreds = OK. Thousands = worry. Millions = split the collection.

4

Don't be afraid of app-side joins

If indexed well, two quick queries can beat a fat aggregation.

5

Consider the read/write ratio

Design for the operation that runs most often, not the rare one.

6

How you model depends on how you access data

There is no 'correct' schema without knowing the query patterns.

Six Useful Schema Patterns

Named recipes for common problems

Bucket

Too many tiny docs (e.g., IoT readings)

→ **Group N per bucket document**

Outlier

One doc has a huge array, others tiny

→ **Flag outliers, store overflow elsewhere**

Computed

Recomputing averages/totals on every read

→ **Store the result, update it on write**

Subset

Document too big, but you only need a bit

→ **Keep hot fields inline, rest in another doc**

Extended Reference

Every join needs the same 2-3 fields

→ **Duplicate those few fields into the referrer**

Schema Versioning

Schema changed, old docs still exist

→ **Add a `schema_version` field, migrate on read**

Anti-Patterns (What NOT to Do)

If you see these in your schema, refactor



Massive embedded arrays

Arrays with thousands of entries slow every read and hit the 16MB document limit.



Unbounded growth

A chat document that grows forever with every message. Use one doc per message instead.



Too many collections

One collection per user, per day, per tenant. Mongo works best with few, well-indexed collections.



Using MongoDB like SQL

Many collections connected by ids with joins everywhere. You're fighting the tool.



Storing dates as strings

You lose range queries and sorting. Always use real Date / ISODate.



No indexes

Queries work until your data grows. Then everything crawls. Plan indexes with queries.

MODULE 06 ★ CRITICAL FOR PROD

Indexing

From COLLSCAN horror to IXSCAN heaven

Why Indexes Matter

The book-index analogy

Without an index (COLLSCAN)



MongoDB reads every document, one by one, to find matches.

10 million docs ≈ 10 million comparisons

→ Slow. Gets slower as data grows.

With an index (IXSCAN)



MongoDB jumps directly to matching docs via a sorted B-tree.

10 million docs ≈ ~24 comparisons

→ Fast. Stays fast as data grows.

INTERVIEW TIP Q: "Why does adding an index help?"

Indexes are sorted B-tree structures. MongoDB binary-searches them instead of scanning every doc. The trade-off: writes become slightly slower because each index must also be updated.

Index Types You Should Know

Different shapes for different queries

Single field

On one key. The simplest index.

```
{ email: 1 }
```

Compound

Multiple keys, one index. Order matters.

```
{ city: 1, age: -1 }
```

Multikey

On array fields — indexes each element.

```
{ tags: 1 }
```

Text

For full-text search on strings.

```
{ bio: "text" }
```

Geospatial

For location queries (2d, 2dsphere).

```
{ loc: "2dsphere" }
```

Hashed

Hash the value — used for sharding.

```
{ userId: "hashed" }
```

Unique

No two docs can have the same value.

```
unique: true
```

TTL

Auto-deletes docs after N seconds.

```
expireAfterSeconds
```

Partial

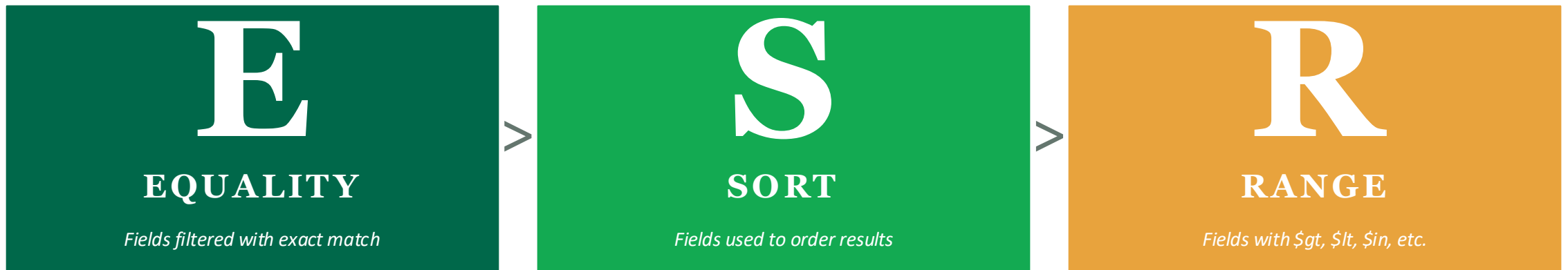
Only indexes docs matching a filter.

```
partialFilterExpression
```


The ESR Rule

How to order keys in a compound index

When building a compound index, the ORDER of keys matters more than anything else. Follow this rule:



Example query:

```
db.users.find({ status: "active", age: { $gt: 18 } }).sort({ joinedAt: -1 })
```

Best index: { status: 1, joinedAt: -1, age: 1 } ← E, S, R

explain() — Reading Query Plans

Your most important debugging tool

Add `.explain("executionStats")` to any query to see what MongoDB actually did:

```
db.users.find({ age: 27 }).explain("executionStats")
```

COLLSCAN

Collection scan — reads every document

totalDocsExamined: 1,000,000

nReturned: 47

executionTimeMs: 820

IXSCAN

Index scan — uses a B-tree directly

totalDocsExamined: 47

nReturned: 47

executionTimeMs: 3

INTERVIEW TIP Q: "How do you know if a query is using an index?"

Run `.explain("executionStats")` and look at the winning plan's stage. IXSCAN = good, using index. COLLSCAN = bad, scanning every document. Compare `totalDocsExamined` vs `nReturned` — ideally they match.

Advanced Concepts

Covered queries, index intersection, and when indexes hurt



Covered Queries

When ALL fields in the query and projection are inside the index, MongoDB never touches the actual documents — the index itself returns the data. Blazing fast.



Index Intersection

MongoDB can combine TWO single-field indexes on the fly. Useful, but a single compound index almost always performs better. Prefer compound.



When Indexes Hurt

Each index speeds up reads but slows down writes (every insert/update must update all indexes). For write-heavy collections or indexes that are rarely used, drop them.



Partial & Sparse

Partial: only indexes docs matching a condition (e.g., active: true). Sparse: only indexes docs where the field exists. Both save space and speed writes.

MODULE 07

Performance & Transactions

Running fast, staying correct

Read & Write Concerns

Trading speed for safety

Write Concern

How many servers must acknowledge a write?

w: 0	Fire and forget — no ack. Risky.
w: 1	Primary ack. Default. Fast.
w: "majority"	Majority of replicas ack. Safer, slower.
j: true	Also wait for write to be on disk journal.

Read Concern

How consistent does the data you read need to be?

"local"	Latest from this node. Default, fast.
"majority"	Only data acked by majority — safe reads.
"linearizable"	Strongest. Slowest. Rare use.
"snapshot"	Point-in-time. For transactions.

INTERVIEW TIP Q: "What is the trade-off with write concerns?"

Higher concern = more servers acknowledge = stronger durability, but slower writes. "majority" is the production default for critical data. w:1 is fine for logs or analytics.

The Profiler & Slow Query Analysis

Find the queries that are hurting you

The database profiler logs operations that take longer than a threshold. Use it to hunt down slow queries.



Off

No profiling. Default.



Slow Only

Log ops slower than slowms (default 100ms).



All Ops

Log everything. Dev only — too noisy for prod.

```
// Enable level-1 profiling (slow queries >= 100ms)
db.setProfilingLevel(1, { slowms: 100 })
// View slowest recent operations
db.system.profile.find().sort({ millis: -1 }).limit(10)
```

INTERVIEW TIP Q: "How would you find slow queries in production?"

Enable the profiler at level 1 (or use Atlas Performance Advisor). Query `db.system.profile` sorted by `millis`. Then run `.explain()` on the worst offenders and add the missing index.

Multi-Document ACID Transactions

All-or-nothing, across multiple documents

Since MongoDB 4.0, you can group multiple writes into one atomic unit: either they all succeed, or none of them do.

A

Atomic

All or nothing

C

Consistent

Valid state → valid state

I

Isolated

Concurrent txs don't see each other

D

Durable

Once committed, it persists

```
const session = client.startSession();
await session.withTransaction(async () => {
  await accounts.updateOne({ _id: A }, { $inc: { bal: -100 } }, { session });
  await accounts.updateOne({ _id: B }, { $inc: { bal: 100 } }, { session });
});
```

INTERVIEW TIP Q: "When should you use MongoDB transactions?"

Only when multiple documents **MUST** update together (e.g., bank transfers between two accounts). If good schema design can keep related data in one document, you don't need a transaction — single-document writes are already atomic in MongoDB.

MODULE 08

Security & MERN

Protect your data • Use MongoDB in a real stack

Field-Level Encryption Basics

Protect individual fields, even from the DBA

Sometimes encrypting the disk isn't enough. You want certain sensitive fields (SSN, card numbers, medical data) to be encrypted INSIDE the document — so even someone with direct database access can't read them.

CSFLE

Client-Side Field-Level Encryption

Encryption happens in the client driver before data is sent. Server never sees plain text. You manage the keys.

Queryable Encryption

Equality queries on encrypted fields

Encrypted data stays encrypted, BUT you can still do equality searches on it. Modern option.

Encryption at Rest

Whole-disk encryption

The underlying storage is encrypted. Protects backup theft but not runtime access. Atlas does this automatically.

INTERVIEW TIP Q: "Why use field-level encryption if data is already encrypted at rest?"

Encryption at rest protects against physical/backup theft but NOT against database admins, compromised apps, or memory dumps. CSFLE encrypts the data in the client — so the DB literally stores ciphertext. Much stronger for sensitive PII.

Why MongoDB in MERN?

The letter M that makes the stack coherent

M

MongoDB

Database

E

Express

Node web framework

R

React

Frontend UI

N

Node.js

JS runtime

Why JavaScript devs love MongoDB:

One language, one data shape

React sends JSON. Express handles JSON. Mongo stores JSON-like docs. No object-relational mapping dance.

Flexible schema matches React state

Your UI often evolves faster than a rigid SQL schema can. MongoDB keeps up.

Mongoose ODM

Adds schemas, validation, middleware, and `.populate()` on top of the driver. Industry standard in Node.js.

Atlas = zero DevOps

MERN developers often ship solo. Atlas removes the 'who manages the DB' problem entirely.

Using MongoDB in Node.js

The native driver vs Mongoose

Native driver (mongodb)

```
import { MongoClient } from "mongodb"
const client = new MongoClient(uri);
await client.connect();

const db = client.db("shop");
const users = db.collection("users");

await users.insertOne({ name: "Aisha" });
```

Mongoose ODM

```
import mongoose from "mongoose"
await mongoose.connect(uri);

const User = mongoose.model("User", {
  name: { type: String, required: true },
  email: { type: String, unique: true }
});
await User.create({ name: "Aisha", email: ... });
```

Schema validation	You do it yourself	Built-in
Middleware hooks	No	Yes (pre/post save, etc.)
Population (joins)	Manual \$lookup	.populate() one-liner
When to use	Simple scripts / perf	Real apps with models

Interview-Ready Cheatsheet

What a beginner MUST be able to answer

Is MongoDB SQL or NoSQL? → NoSQL, document-oriented

Insert one vs many? → insertOne, insertMany (faster in batches)

What is ObjectId? → 12-byte auto unique id (timestamp + random + counter)

COLLSCAN vs IXSCAN? → Scan all docs vs use index — check with .explain()

findOne vs find? → findOne returns one doc; find returns a cursor

What is Mongoose? → ODM on top of the driver; adds schemas & validation

BSON vs JSON? → BSON is binary JSON with extra types (Date, ObjectId, Decimal128)

Embed vs reference? → Embed when read together & bounded; reference when big/shared

Compound index order? → ESR: Equality, Sort, Range

When use transactions? → Only when multi-doc updates must be atomic

Pagination? → skip+limit is simple but slow; range on indexed sort key scales

Why MongoDB in MERN? → Same JSON shape end-to-end, flexible schema, fast

You now have the foundation

Go build things.

Next steps:

1. Create a free Atlas cluster and connect from Compass.
2. Build a small Express + Mongoose API (e.g., a task manager).
3. Take MongoDB University's M001 — it's free and excellent.
4. Revisit this deck before every interview.

Happy building 🌿